

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#02)

Adders, Decoders, Mux

Student Name: Shafay Ul Hassan Roll No: 24i-6552 Section: A

Date performed: 25-08, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1a:

Description:

In this task we implemented half adder using both gate level and data level approach. We make separate modules for the sum and carry and then verify them by using test bench.

Code:

Data Level:

```
module sum(a,b,out);
input a;
input b;
output out;
assign out=a^b;
endmodule
module carry(a,b,c);
input a;
input b;
output c;
assign c=a&b;
endmodule
module test_bench_half_adder();
reg x;
reg y;
wire out1;
wire c1;
sum s1(.a(x),.b(y),.out(out1));
carry c2(.a(x),.b(y),.c(c1));
initial
begin
x=0;y=0;#50;
x=0;y=1;#50;
x=1;y=0;#50;
x=1;y=1;#50;
end
endmodule
```

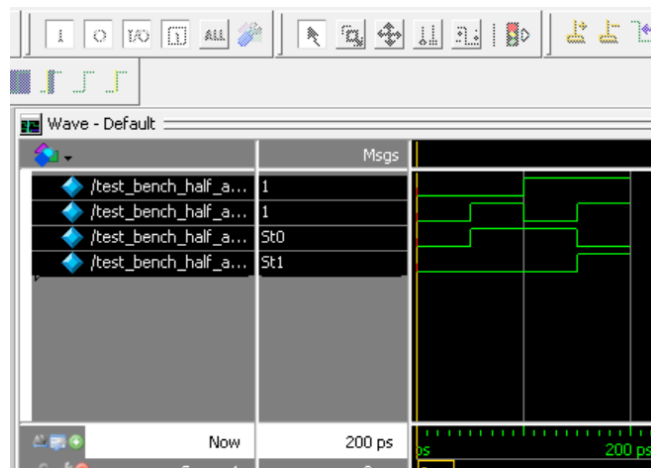
Gate Level:

```
module sum(a,b,out);
input a;
input b;
output out;
xor(out,a,b);
endmodule
module carry(a,b,c);
input a;
input b;
output c;
and(c,a,b);
endmodule
module test_bench_half_adder();
reg x;
```

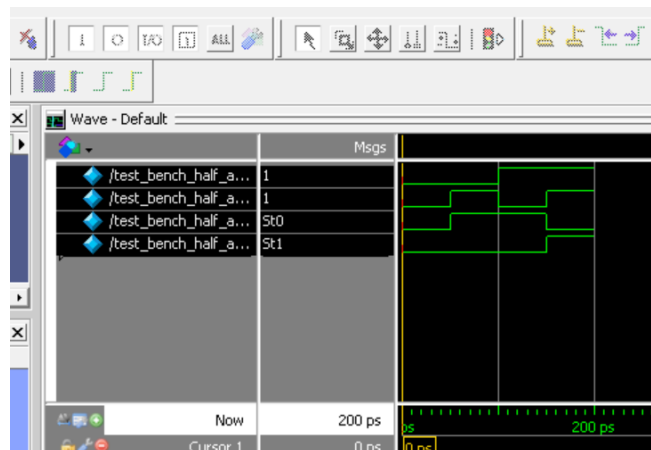
```
reg y;
wire out1;
wire c1;
sum s1(.a(x),.b(y),.out(out1));
carry c2(.a(x),.b(y),.c(c1));
initial
begin
x=0;y=0;#50;
x=0;y=1;#50;
x=1;y=0;#50;
x=1;y=1;#50;
end
endmodule
```

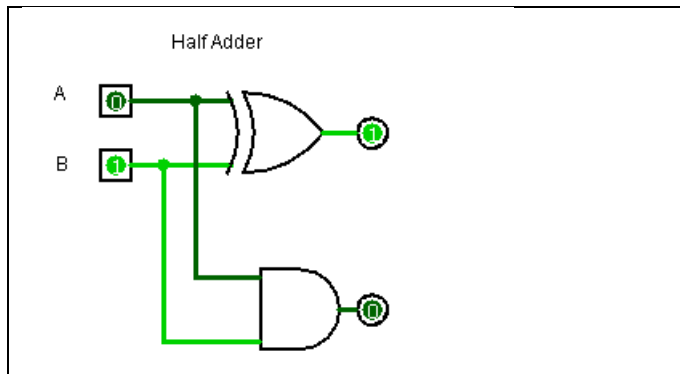
Output:

Dataflow Level:



Gate Level:





Task 1b:

Description:
In this task we implemented full adder using both gate level and data level approach. We make separate modules for the sum and carry and then verify them by using test bench.
Code:
<pre>Dataflow Level: module sum(a,b,c,out); input a; input b; input c; output out; assign out=a^b^c; endmodule module carry(a,b,c,car); input a; input b; input c; output car; assign car=(a&b) (b&c) (a&c); endmodule module test_bench_full_adder(); reg x; reg y; reg z; wire out1; wire c1; sum s1(.a(x),.b(y),.c(z),.out(out1)); carry c2(.a(x),.b(y),.c(z),.car(c1)); initial begin x=0;y=0;z=0;#50; x=0;y=0;z=1;#50; x=0;y=1;z=0;#50; x=0;y=1;z=1;#50; x=1;y=0;z=0;#50; x=1;y=0;z=1;#50; x=1;y=1;z=0;#50; x=1;y=1;z=1;#50; end endmodule Gate Level: module sum(a,b,c,out); input a; input b; input c; output out; xor(out,a,b,c); endmodule module carry(a,b,c,car);</pre>

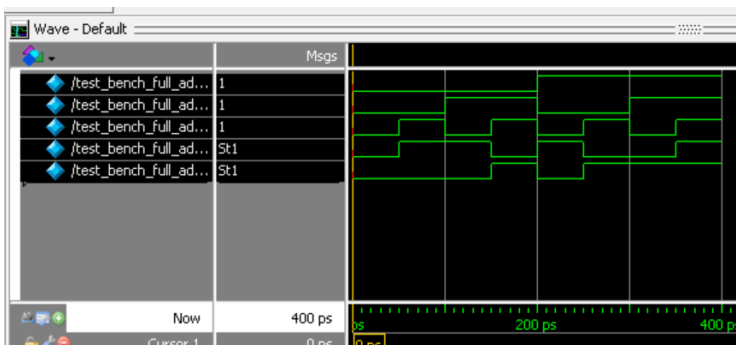
```

input a;
input b;
input c;
wire s1;
wire s2;
wire s3;
output car;
and(s1,a,b);
and(s2,b,c);
and(s3,c,a);
or(car,s1,s2,s3);
endmodule
module test_bench_full_adder1();
reg x;
reg y;
reg z;
wire out1;
wire c1;
sum s1(.a(x),.b(y),.c(z),.out(out1));
carry c2(.a(x),.b(y),.c(z),.car(c1));
initial
begin
x=0;y=0;z=0;#50;
x=0;y=0;z=1;#50;
x=0;y=1;z=0;#50;
x=0;y=1;z=1;#50;
x=1;y=0;z=0;#50;
x=1;y=0;z=1;#50;
x=1;y=1;z=0;#50;
x=1;y=1;z=1;#50;
end
endmodule

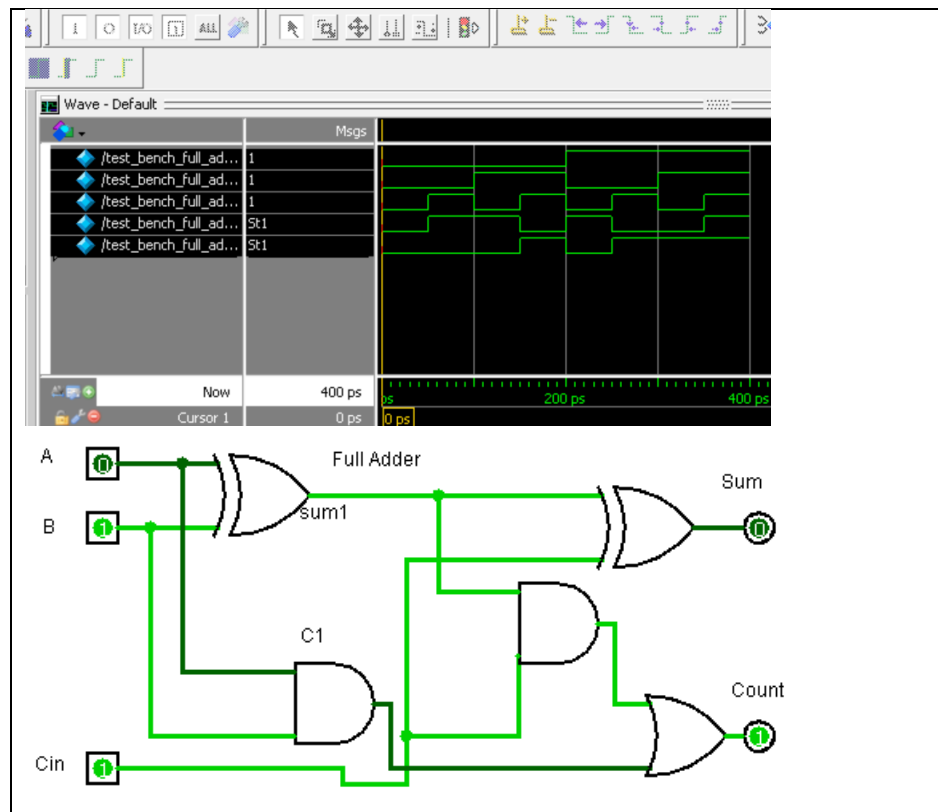
```

Output:

Gate Level:



Dataflow Level:



Task 2:

Description:

In this task we implemented 3*8 decoder using both gate level and data level approach. We took 3 inputs and form combinations of that inputs and then allot them in such a way that at one combination of inputs there will be one output that would be high.

Code:

Gate Level:

```
module decoder(a,b,c,out1,out2,out3,out4,out5,out6,out7,out8);
input a,b,c;
output out1,out2,out3,out4,out5,out6,out7,out8;
wire s1;
wire s2;
wire s3;
not(s1,a);
not(s2,b);
not(s3,c);
and a1(out1, s1, s2, s3);
and a2(out2,s1,s2,c);
and a3(out3,s1,b,s3);
and a4(out4,s1,b,c);
and a5(out5,a,s2,s3);
and a6(out6,a,s2,c);
```

```
and a7(out7,a,b,s3);
and a8(out8,a,b,c);
endmodule
module decoder_testbench();
reg x,y,z;
wire c1,c2,c3,c4,c5,c6,c7,c8;
decoder
uu(.a(x),.b(y),.c(z),.out1(c1),.out2(c2),.out3(c3),.out4(c4),.out5(c5),.out6(c6)
,.out7(c7),.out8(c8));
initial
begin
x=0;y=0;z=0;#50;
x=0;y=0;z=1;#50;
x=0;y=1;z=0;#50;
x=0;y=1;z=1;#50;
x=1;y=0;z=0;#50;
x=1;y=0;z=1;#50;
x=1;y=1;z=0;#50;
x=1;y=1;z=1;#50;
end
endmodule
```

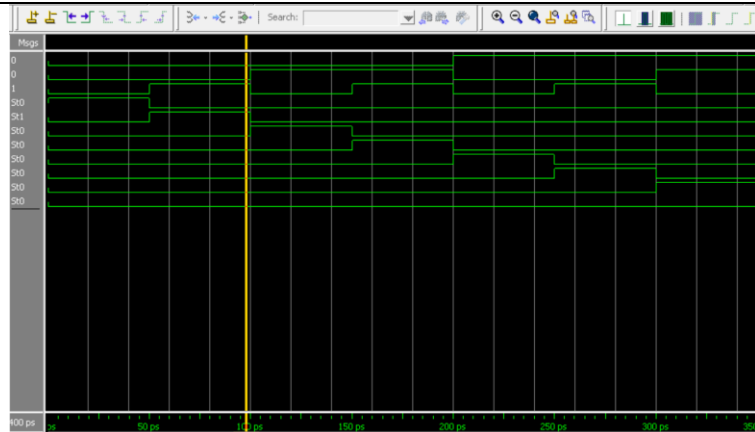
DataFlow Level:

```
module decoder(a,b,c,out1,out2,out3,out4,out5,out6,out7,out8);
input a,b,c;
output out1,out2,out3,out4,out5,out6,out7,out8;
wire s1;
wire s2;
wire s3;
assign s1=~a;
assign s2=~b;
assign s3=~c;
assign out1=(s1&s2)&s3;
assign out2=(s1&s2)&c;
assign out3=(s1&b)&s3;
assign out4=(s1&b)&c;
assign out5=(a&s2)&s3;
assign out6=(a&s2)&c;
assign out7=(a&b)&s3;
assign out8=(a&b)&c;
endmodule
module decoder_test_bench();
reg x,y,z;
wire c1,c2,c3,c4,c5,c6,c7,c8;
decoder
uu(.a(x),.b(y),.c(z),.out1(c1),.out2(c2),.out3(c3),.out4(c4),.out5(c5),.out6(c6)
,.out7(c7),.out8(c8));
initial
begin
x=0;y=0;z=0;#50;
x=0;y=0;z=1;#50;
x=0;y=1;z=0;#50;
x=0;y=1;z=1;#50;
x=1;y=0;z=0;#50;
x=1;y=0;z=1;#50;
x=1;y=1;z=0;#50;
x=1;y=1;z=1;#50;
end
```

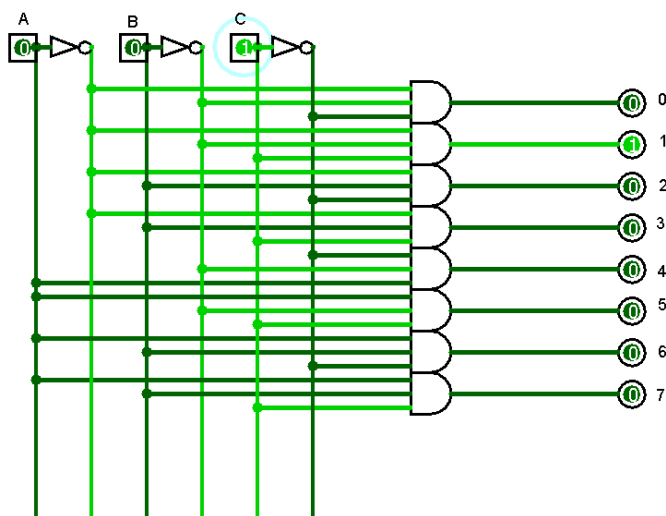
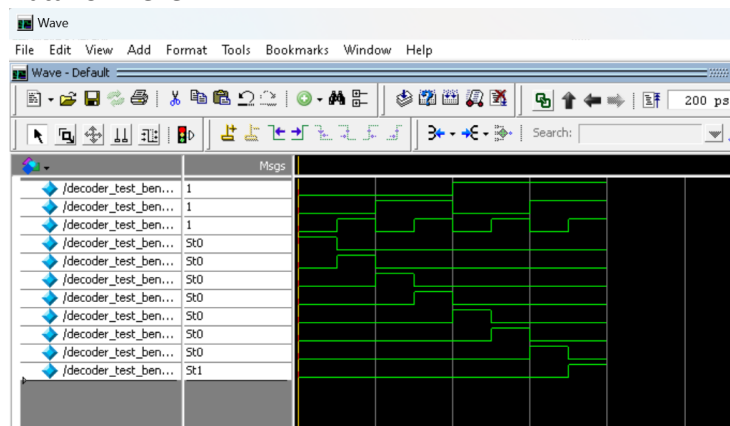

endmodule

Output:

Gate LEVEL:



Dataflow Level:



Task 3:

Description:

In this task we implemented 8*1 muxr using both gate level and data level approach. We took 8 inputs and 3 select lines form combinations of that inputs and then alot them in such a way that at one combination of inputs there will be one output that would be high.

Code:

Gate Level:

```
module mux(a,b,c,d,e,f,g,h,i,j,k,out);
input a,b,c,d,e,f,g,h,i,j,k;
output out;
wire s1,s2,s3,d1,e1,f1,g1,h1,i1,j1,k1;
not(s1,a);
not(s2,b);
not(s3,c);
and(d1,s1,s2,s3,d);
and(e1,s1,s2,c,e);
and(f1,s1,b,s3,f);
and(g1,s1,b,c,g);
and(h1,a,s2,s3,h);
and(i1,a,s2,c,i);
and(j1,a,b,s3,j);
and(k1,a,b,c,k);
or(out,d1,e1,f1,g1,h1,i1,j1,k1);
endmodule

module mux1_test_bench();
reg x,y,z,p,q,r,s,t,u,v,w;
wire c1;
mux uu(.a(x),.b(y),.c(z),.d(p),.e(q),.f(r),.g(s),.h(t),.i(u),.j(v),.k(w),.out(c1));
initial
begin
p = 0;q = 1;r = 0;s = 1;t = 1;u = 0;v = 1;w = 0;
x=0;y=0;z=0;#50;
x=0;y=0;z=1;#50;
x=0;y=1;z=0;#50;
x=0;y=1;z=1;#50;
x=1;y=0;z=0;#50;
x=1;y=0;z=1;#50;
x=1;y=1;z=0;#50;
x=1;y=1;z=1;#50;
end
endmodule
```

Dataflow Level:

```
module mux(a,b,c,d,e,f,g,h,i,j,k,out);
input a,b,c,d,e,f,g,h,i,j,k;
output out;
wire s1,s2,s3,d1,e1,f1,g1,h1,i1,j1,k1;
not(s1,a);
not(s2,b);
not(s3,c);
and(d1,s1,s2,s3,d);
and(e1,s1,s2,c,e);
and(f1,s1,b,s3,f);
and(g1,s1,b,c,g);
```

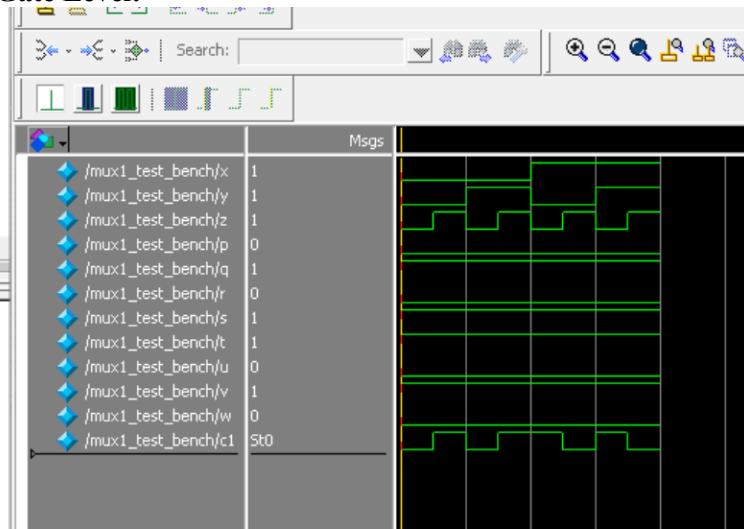
```

and(h1,a,s2,s3,h);
and(i1,a,s2,c,i);
and(j1,a,b,s3,j);
and(k1,a,b,c,k);
or(out,d1,e1,f1,g1,h1,i1,j1,k1);
endmodule
module mux1_test_bench();
reg x,y,z,p,q,r,s,t,u,v,w;
wire c1;
mux uu(.a(x),.b(y),.c(z),.d(p),.e(q),.f(r),.g(s),.h(t),.i(u),.j(v),.k(w),.out(c1));
initial
begin
p = 0;q = 1;r = 0;s = 1;t = 1;u = 0;v = 1;w = 0;
x=0;y=0;z=0;#50;
x=0;y=0;z=1;#50;
x=0;y=1;z=0;#50;
x=0;y=1;z=1;#50;
x=1;y=0;z=0;#50;
x=1;y=0;z=1;#50;
x=1;y=1;z=0;#50;
x=1;y=1;z=1;#50;
end
endmodule module mux(a,b,c,d,e,f,g,h,i,j,k,out);
input a,b,c,d,e,f,g,h,i,j,k;
output out;
wire s1,s2,s3,d1,e1,f1,g1,h1,i1,j1,k1;
assign s1=~a;
assign s2=~b;
assign s3=~c;
assign d1=(s1&s2)&(s3&d);
assign e1=(s1&s2)&(c&e);
assign f1=(s1&b)&(s3&f);
assign g1=(s1&b)&(c&g);
assign h1=(a&s2)&(s3&h);
assign i1=(a&s2)&(c&i);
assign j1=(a&b)&(s3&j);
assign k1=(a&b)&(c&k);
assign out=d1|e1|f1|g1|h1|i1|j1|k1;
endmodule
module mux_test_bench();
reg x,y,z,p,q,r,s,t,u,v,w;
wire c1;
mux uu(.a(x),.b(y),.c(z),.d(p),.e(q),.f(r),.g(s),.h(t),.i(u),.j(v),.k(w),.out(c1));
initial
begin
p = 0;q = 1;r = 0;s = 1;t = 1;u = 0;v = 1;w = 0;
x=0;y=0;z=0;#50;
x=0;y=0;z=1;#50;
x=0;y=1;z=0;#50;
x=0;y=1;z=1;#50;
x=1;y=0;z=0;#50;
x=1;y=0;z=1;#50;
x=1;y=1;z=0;#50;
x=1;y=1;z=1;#50;
end
endmodule

```

Output:

Gate Level:



Dataflow Level:

